

Employee Management System - Linked list

05 September 2026 17:45

Creation of the employee - blueprints

```
class Employee {
public:
    int id;
    string name;
    float salary;
    Employee* next;

    Employee(int i, string n, float s) {
        id = i;
        name = n;
        salary = s;
        next = NULL;
    }
};
```

OOPS

constructor

C - prog - OOPS, (python, c++, java) → object.

* technique DSA → Live projects, CRUD operations.

Stack.

CRUD

create ↓ Read ↓ update

Delete.

* javascript
* React
* Mobile.

Creation of the employee - management

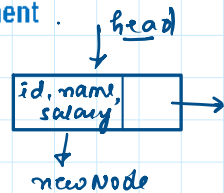
```
class EmployeeManagement {
    Employee* head = NULL;

public:
    // CREATE method
    void create(int id, string name, float salary) {
        Employee* newNode = new Employee(id, name, salary);

        if (head == NULL) {
            head = newNode;
            return;
        }

        Employee* temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
};
```

method



Linked List

OOPS.

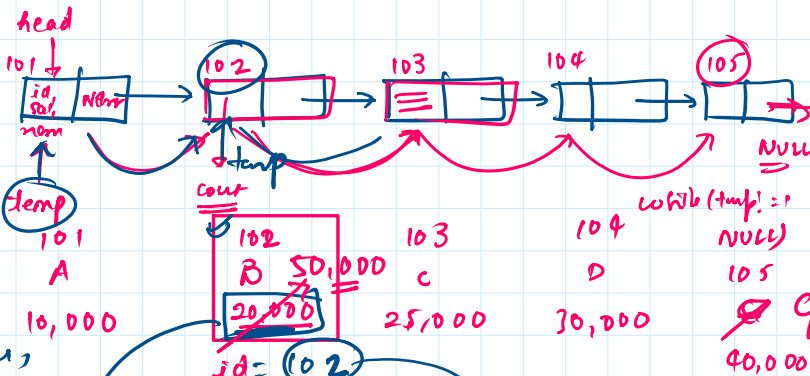
* INSERT INTO emp VALUES (id, name, salary);

```
// READ method
void read() {
    Employee* temp = head;

    while (temp != NULL) {
        cout << temp->id << " "
              << temp->name << " "
              << temp->salary << endl;
        temp = temp->next;
    }
}
```

method

SELECT *
FROM
Employee;



```
// UPDATE method
void update(int id, float newSalary) {
    Employee* temp = head;

    while (temp != NULL) {
        if (temp->id == id) {
            temp->salary = newSalary;
            return;
        }
        temp = temp->next;
    }
}
```

method

UPDATE

if (temp->id == id) {
temp->salary = newSalary;
}

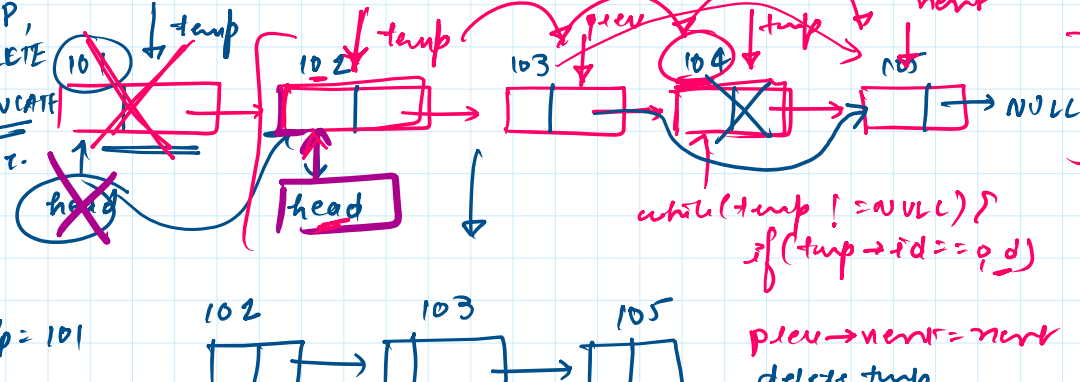
```
// DELETE method
void remove(int id) {
    if (head == NULL) return;

    // Delete first node
    if (head->id == id) {
        Employee* temp = head;
        head = head->next;
        delete temp;
        return;
    }

    Employee* temp = head;
```

method

DELETE



```

return;
}

Employee* temp = head;

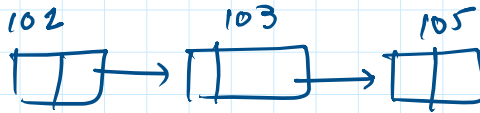
while (temp->next != NULL &&
temp->next->id != id) {
temp = temp->next;
}

if (temp->next != NULL) {
Employee* del = temp->next;
temp->next = del->next;
delete del;
}
}

```

idp = 101

can 2



prev->next = next
delete temp

```

in main()
EmployeeManagement emp;

// CREATE
emp.create(101, "Rahul", 50000);
emp.create(102, "Priya", 60000);
emp.create(103, "Amit", 55000);

// READ
cout << "Employees\n";
emp.read();

// UPDATE
emp.update(102, 70000);

cout << "After Update\n";
emp.read();

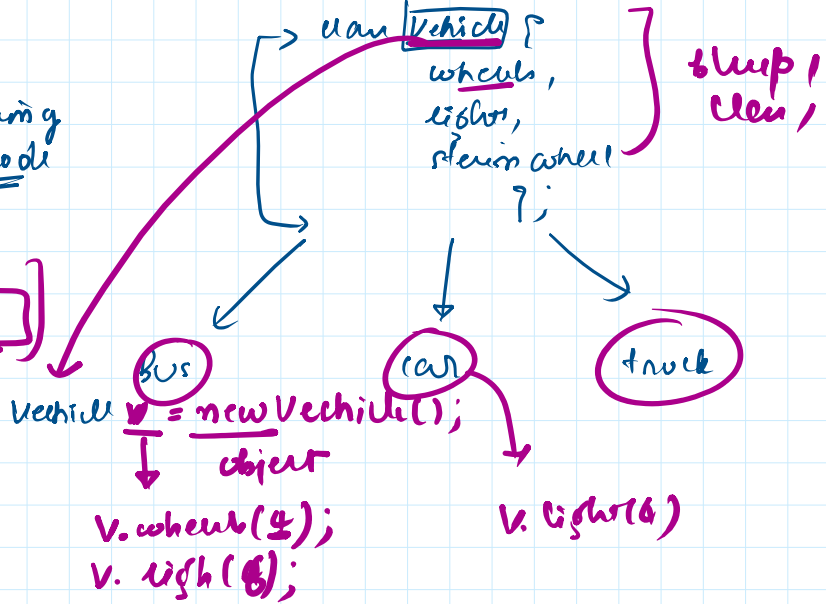
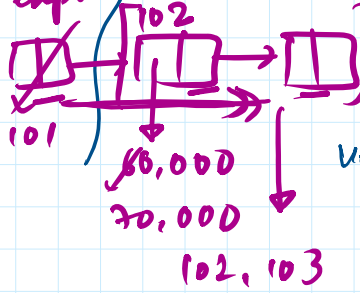
// DELETE
emp.remove(101);

cout << "After Delete\n";
emp.read();

return 0;
}

```

object creation
According
driver code

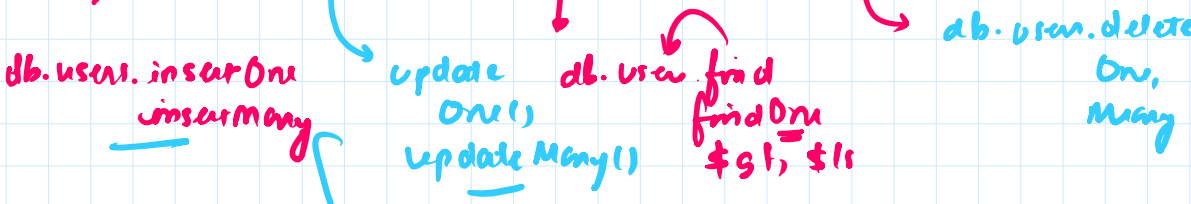


From <<https://mail.google.com/mail/u/0/#inbox/KtbxLrjGNgXSDhVdhSsNjKvNNGPmwcDzfl>>

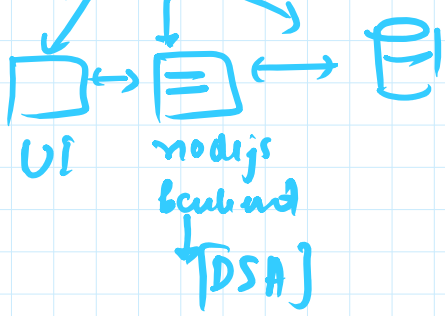
mongodb - NO SQL

postgres, supabase, firebase

CREATE, UPDATE, READ, DELETE.



FULL software En. principle.



15 - 1 = 14
14 - 2 = 12
12 - 4 = 8
8 - 5 = 3
3 - 1 = 2
2 - 2 = 0

2kg left
bags

* 15kg (capacity)
bags

* fraction values as well.

max profit (54.6)

(15)

weight

if (calculated value $\sum x_i w_i$ = knapsack (cap) $\times$)

op → max profit (54.6)

$\sum x_i w_i \leq 15$
15 ≤ 15

(from list) objets (o) → 1, 2, 3, 4, 5, 6, 7

(Kupen) profits (p) → 10, 5, 15, 7, 6, 18, 3

(1kg, 2kg) weight (w) → 2, 3, 5, 7, 1, 4, 1

max value P/w

10	5	15	7	6	18	3
5	1.5	3	1	1.5	4.5	3

(x) (x₁ x₂ x₃ x₄ x₅ x₆ x₇)

(x) (1 2/3 1 0 1 1 1)

profit → $\sum x_i p_i \Rightarrow 1 \times 10 + 5 \times 2/3 + 1 \times 15 + 0 \times 7 + 1 \times 6 + 1 \times 18 + 1 \times 3$

weight → $\sum x_i w_i \Rightarrow 1 \times 2 + 3 \times 2/3 + 5 \times 1 + 0 \times 7 + 1 \times 1 + 1 \times 4 + 1 \times 1$

(15)

Greedy technique

Job selection

0/1

Algorithm